

CricketIQ

IPL Analytics & Predictive Intelligence Platform

Project Documentation

Requirement Gathering · User Stories · UI Design · UML Diagrams · Table Design & Normalisation

Submitted to	[Guide Name], [Designation]
Submitted by	[Student Name(s)] · [Register / Roll Number]
Department	[Department Name]
Institution	[Institution / College Name]
Academic Year	[Academic Year]
Repository	https://github.com/arjuntp-397/Mini-Project-2

Table of Contents

1. Requirement Gathering
2. User Story Creation
3. UI Design
4. UML Diagrams
5. Table Design
6. Design: Customisation on Templates
7. Table Normalisation

1. Requirement Gathering

1.1 Purpose

CricketIQ is a web analytics platform for the IPL: one place to browse team, player and match records, view season analytics, and run four prediction tools — win probability, player form forecast, playing-XI suggestion, and player similarity.

1.2 Scope

In scope:

- Team, player and match directories with detail pages.
- Analytics dashboard: win trends, run and wicket leaderboards.
- Four intelligence tools: win probability, form forecast, playing-XI, similarity.
- Dark/light themes, responsive from mobile to desktop.

Out of scope: live ball-by-ball ingestion, user accounts/login, payments.

1.3 Functional Requirements

ID	Requirement	Route
FR-01	Browse teams (paginated, searchable).	GET /teams
FR-02	Team profile: squad + latest season record.	GET /teams/<id>
FR-03	Browse players, filter by role.	GET /players
FR-04	Player profile: career totals + full record.	GET /players/<id>
FR-05	Browse matches, filter by season.	GET /matches
FR-06	Match result + full scorecard.	GET /matches/<id>
FR-07	Win-trend charts + leaderboards.	GET /analytics
FR-08	Simulate a matchup, view win probability.	GET /api/predict/match
FR-09	Forecast a player's next-innings runs.	GET /api/predict/player
FR-10	Suggested playing XI for a matchup.	GET /api/assistant/team_selection
FR-11	Find statistically similar players.	GET /api/players/similarity
FR-12	Client-side filter, search, sort on any list.	client JS
FR-13	Persisted dark / light theme.	client JS

1.4 Non-Functional Requirements

Category	Requirement
Performance	ML models cached in-process; eager-loaded joins avoid N+1 queries.
Usability	One component set across all pages; inline errors + toasts, no alert().

Category	Requirement
Accessibility	Contrast $\geq 4.5:1$ body / $\geq 3:1$ large text; visible focus ring; ARIA-live regions.
Responsiveness	Sidebar $\geq 1200\text{px}$, drawer below; verified 390px–1440px.
Maintainability	Template inheritance + shared macros; one CSS token layer.
Reliability	Defined empty/error state on every list and API call.

2. User Story Creation

Written from the perspective of an Analyst — the system's single actor; CricketIQ has no login or roles.

ID	User story	Acceptance criteria
US-01	Browse all teams to see franchise details at a glance.	Paginated, filterable directory; each card links to a team page.
US-02	View a team's squad and season record to assess form.	Sortable squad table + season stat tiles.
US-03	Filter players by role to focus on a position.	Role dropdown + instant on-page search.
US-04	View a player's career totals and match record.	Career tiles + batting/bowling tables; empty state if none.
US-05	Browse matches for a season to review fixtures.	Season filter; sortable, links to each scorecard.
US-06	View a match scorecard for the full breakdown.	Both innings' tables; empty state if no scorecard.
US-07	See win trends and leaderboards to spot form.	Trend chart (or ranked bars for one season) + two leaderboards.
US-08	Simulate a matchup for a win-probability estimate.	Rejects invalid pairs inline; doughnut chart + factors on success.
US-09	Forecast a player's next-innings runs.	Recent form, projected runs, confidence label.
US-10	Get a suggested playing XI for a matchup.	11 players with role breakdown + reasoning.
US-11	Find players statistically similar to one chosen.	Cluster ID + up to 5 similar players.
US-12	Switch dark / light theme to match my environment.	Choice persists; no flash of wrong theme.

3. UI Design

The interface uses a custom design system, "**Floodlight**" — one CSS token layer (static/css/style.css) over the Bootstrap 5 grid, colour in OKLCH throughout. Dark is default; light mode is fully supported, both verified for contrast rather than assumed. Pitch-green and analytics-blue — the reflex choices for a cricket dashboard — were deliberately avoided in favour of a cool graphite surface with a single amber accent.

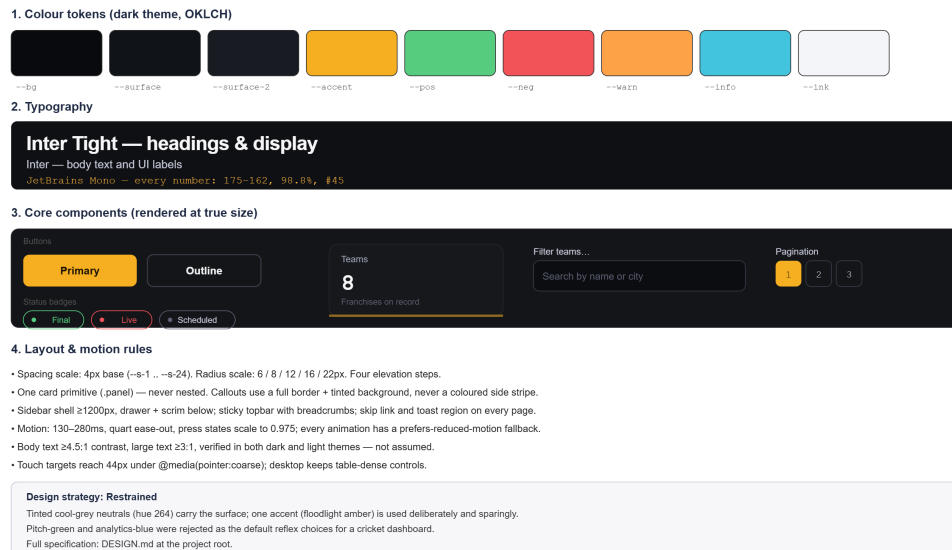


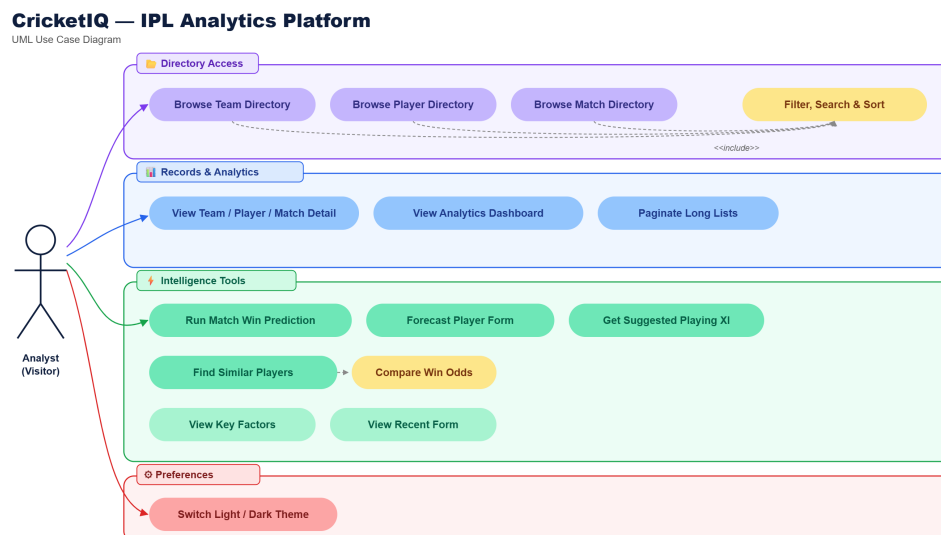
Figure 3.1 — Colour tokens, typography and core components, rendered at true size.

Type pairs three families: Inter Tight (headings), Inter (body), JetBrains Mono (every number, so table figures never jitter). Full spec in DESIGN.md.

4. UML Diagrams

4.1 Use Case Diagram

One actor — an unauthenticated analyst — grouped into four service areas. The three directory use cases share a single "Filter, Search & Sort" behaviour via <<include>>.



CricketIQ · Use Case Diagram

Figure 4.1 — Use case diagram.

4.2 Class Diagram

Condensed view of the six SQLAlchemy models. Team is the hub, referenced by Player and by Match in four FK roles (home, away, winner, toss winner). Batting and bowling figures live in their own tables rather than as columns on Player — see Section 7.

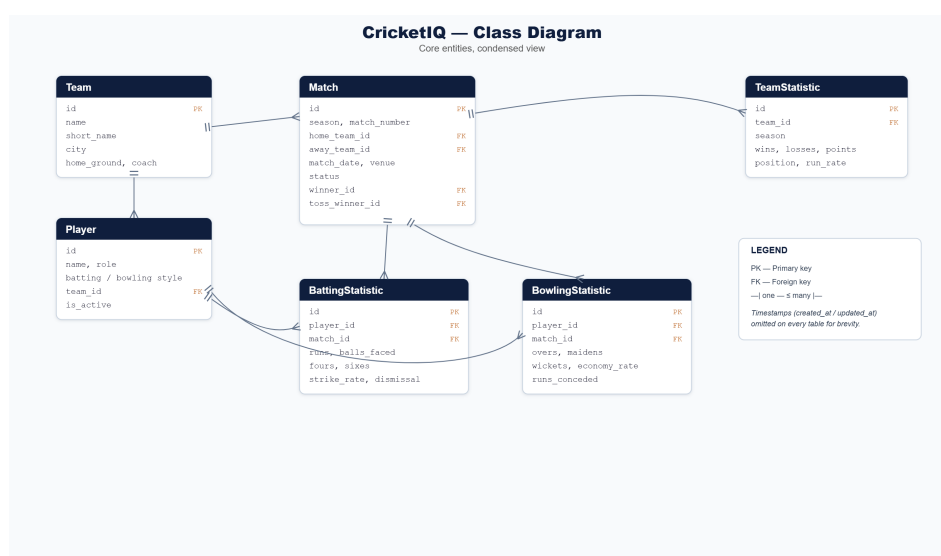


Figure 4.2 — Class diagram.

4.3 Process Flow — Match Win Prediction

End-to-end flow for the win-probability use case, across Browser, Flask API and Model/Data. Errors return inline plus a toast, never a blocking alert().

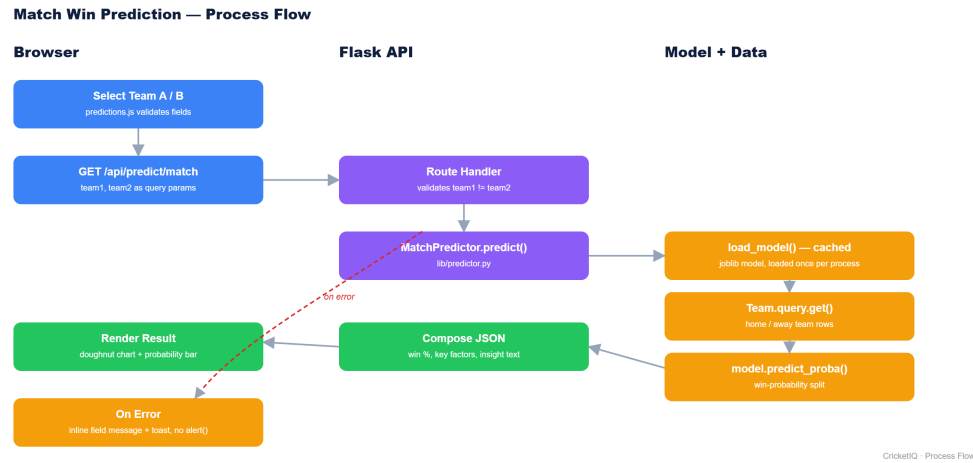


Figure 4.3 — Match prediction process flow.

5. Table Design

Six tables, implemented with Flask-SQLAlchemy. Full field list, types and keys below; every table also carries created_at / updated_at, omitted here for brevity.

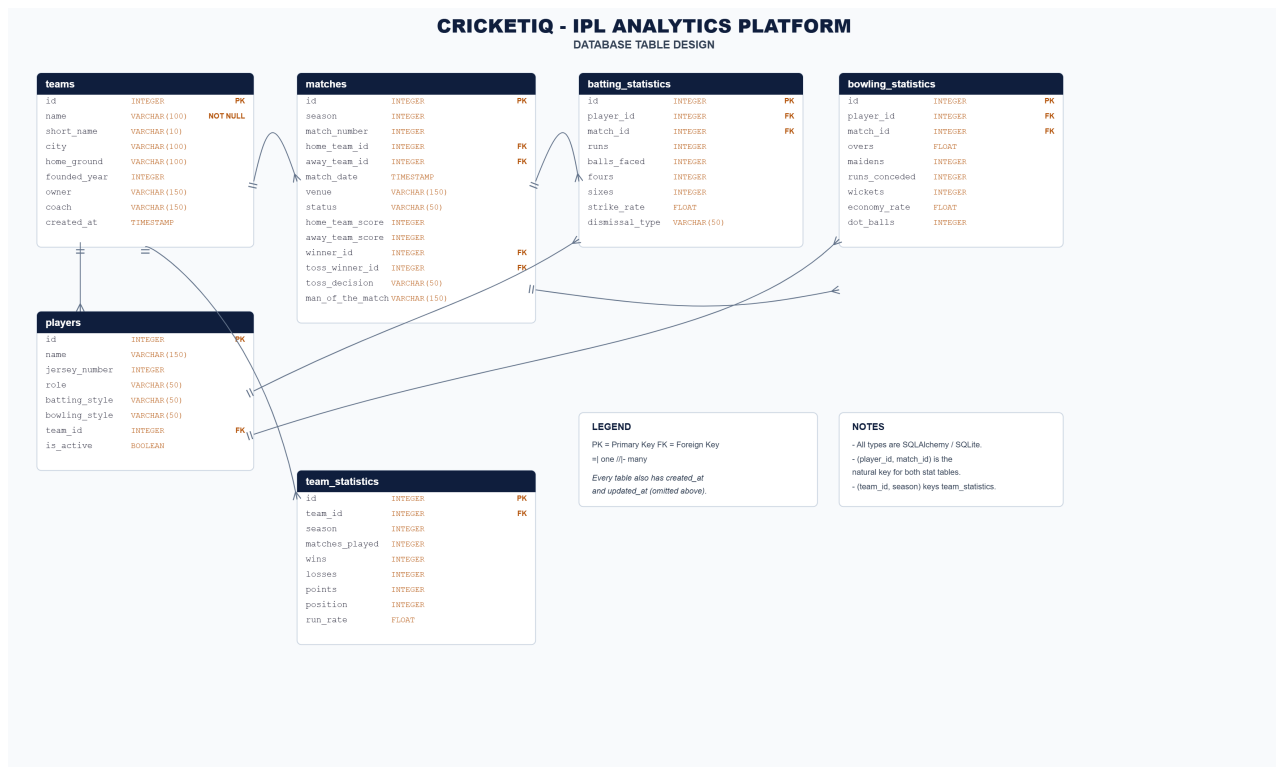


Figure 5.1 — Database table design.

Team is the reference point for Player (squad) and Match (home/away/winner/toss, four FK roles). BattingStatistic and BowlingStatistic each key off (player_id, match_id); TeamStatistic keys off (team_id, season).

6. Design: Customisation on Templates

All 13 pages extend one base layout (templates/base.html), which owns the sidebar, topbar, footer, toast region and five Jinja2 blocks. Each child template overrides only the blocks it needs.

Two macro files remove repetition further: `_icons.html` (one SVG sprite, no Font Awesome dependency) and `_components.html` (pagination, empty states, badges, filter search — shared by every listing page).

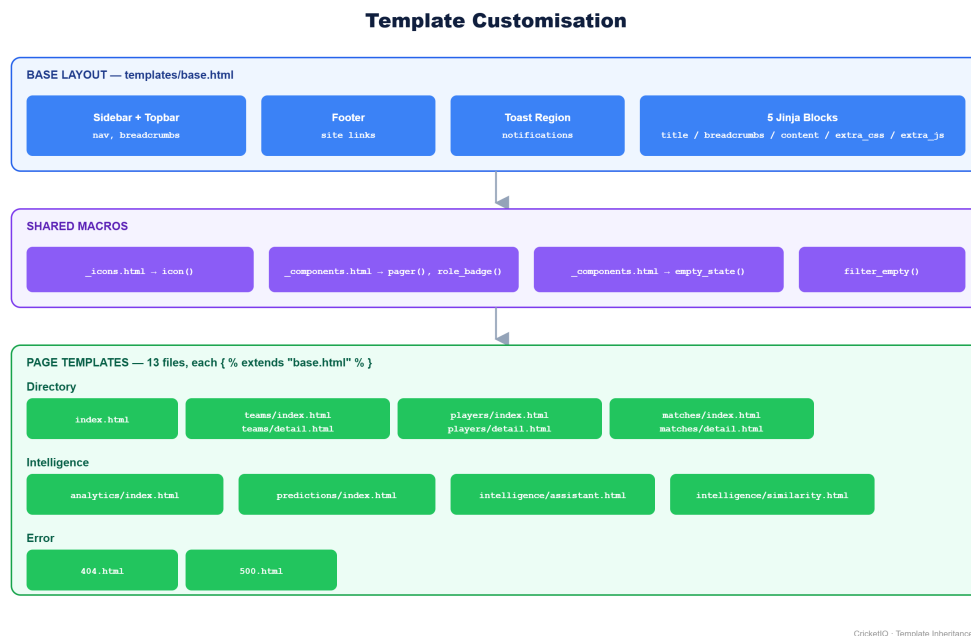


Figure 6.1 — Template inheritance.

Example child template:

```
{% extends "base.html" %}
{% block title %}Teams — CricketIQ{% endblock %}
{% block content %}
... page markup, using icon() and role_badge() macros ...
{% endblock %}
```

7. Table Normalisation

7.1 1NF

Every column is atomic; no repeating groups. A player's many innings become rows in `batting_statistics` / `bowling_statistics`, not numbered columns on players.

7.2 2NF

All six tables key on a single-column id, so 2NF holds automatically. The real business keys — `(player_id, match_id)` and `(team_id, season)` — have every other column depend on the full pair, never on one half.

7.3 3NF

No column depends on another non-key column. A player's team name is never copied onto players — only `team_id`, resolved via FK. Season aggregates (`wins`, `points`, `run_rate`) live in `team_statistics`, not on teams, since they describe a `(team, season)` pair and would otherwise be overwritten every season.

7.4 Summary

Table	1NF	2NF	3NF	Notes
teams	Yes	Yes	Yes	No repeating groups; no transitive dependency.
players	Yes	Yes	Yes	<code>team_id</code> is an FK, not a copied team name.
matches	Yes	Yes	Yes	Four team roles are FKs.
batting_statistics	Yes	Yes	Yes	One row per (player, match).
bowling_statistics	Yes	Yes	Yes	One row per (player, match).
team_statistics	Yes	Yes	Yes	Keyed by (team, season), off teams.

Further detail: `README.md`, `PROJECT_OVERVIEW.md`, `DESIGN.md` at the project root; source at <https://github.com/arjuntp-397/Mini-Project-2>.